
MEMORIA DESCRIPTIVA

Millán Merino Martínez



GymProgress

19 DE MAYO DE 2026

PROYECTO FINAL

CIFP CAMINO DE LA MIRANDA

Contenido

Introducción	2
Resumen.....	2
Justificación del proyecto.....	3
Objeto del proyecto	3
Desarrollo	4
Lenguajes empleados.....	5
Despliegue y distribución del sistema.....	6
Requisitos de los clientes	7
Licenciamiento	8
Recursos	8
Hardware:.....	8
Software	9
Humanos	10
Previsión económica del coste del proyecto	10
Descripción de la aplicación	12
Funcionamiento general	12
Arquitectura	15
Interfaz	23
Autoevaluación y conclusiones	25

Introducción

Resumen

El presente proyecto describe el diseño, desarrollo e implementación de “GymProgress”, un sistema informático integral con arquitectura cliente-servidor destinado a la gestión, planificación y monitorización del entrenamiento de fuerza y composición corporal. Su motivación proporcionar a los usuarios una herramienta centralizada rápida y precisa frente a los métodos registro ineficientes actuales, incorporando variables avanzadas de autorregulación como la Escala de Esfuerzo Percibido (RPE).

A nivel técnico, el sistema se divide en un servidor (Backend) desarrollador como una API RESTful en Java utilizando el entorno de trabajo Spring Boot, conectado a una base de datos relacional MySQL normalizada. Por otro lado, el cliente (Frontend) es una aplicación móvil nativa para Android programada en Kotlin, diseñada priorizando la usabilidad mediante directrices Material Design y adaptabilidad para entornos de gimnasio (como el modo oscuro automático y teclados contextuales). Esta herramienta permite registrar entrenamientos en tiempo real, visualizar historiales gráficos y gestionar catálogos de ejercicios mediante distintos roles de acceso.

Como conclusiones generales, abordar el desarrollo completo (Full-Stack) representó un reto significativo, especialmente en la integración de ambos entornos y en la implementación de seguridad mediante tokens JWT. No obstante, se ha logrado una aplicación robusta y escalable gracias a la correcta separación de la lógica de presentación y negocio. Este proyecto ha permitido comprender el ciclo de vida íntegro de un producto de software, demostrando que un diseño minimalista mejora sustancialmente la experiencia del usuario final.

Palabras clave: Entrenamiento de fuerza, Arquitectura Cliente-Servidor, Android Nativo, Kotlin, Java, Spring Boot, API RESTful, Base de Datos Relacional, Usabilidad.

Justificación del proyecto

La elección de desarrollar "GymProgress" como proyecto final nace de la convergencia de dos grandes intereses personales: el mundo del entrenamiento de fuerza y el desarrollo de software.

En el ámbito deportivo, la motivación principal surge de una creciente concienciación sobre la importancia del cuidado físico y la aplicación de principios científicos para lograr resultados, tales como la sobrecarga progresiva y el control de la fatiga. La gran mayoría de usuarios de salas de musculación registra sus progresos de manera ineficiente. Lo habitual es depender de libretas de papel, aplicaciones genéricas que no se adaptan a las dinámicas reales de un entrenamiento o, directamente, no llevar ningún tipo de registro.

Si bien es cierto que existen en el mercado herramientas consolidadas que abordan este nicho (como Strong, Hevy o FitNotes), mi experiencia como usuario me ha demostrado que a menudo presentan interfaces sobrecargadas con funcionalidades secundarias que distraen del entrenamiento. Además, suelen restringir características fundamentales, como el acceso al historial completo o la analítica detallada del progreso, detrás de modelos de suscripción económica. "GymProgress" busca solucionar este problema ofreciendo una alternativa ágil, directa y con una interfaz "a prueba de fatiga", centrada puramente en registrar cada métrica de forma intuitiva durante la propia sesión.

El objetivo académico de este proyecto es abarcar el ciclo de vida completo de un producto de software real. Construir una arquitectura Full-Stack desde cerro, desarrollando una API RESTful propia en Java con Spring Boot, gestionando la persistencia de datos y la seguridad, y conectándola a una aplicación nativa en Android con Kotlin.

Objeto del proyecto

Mi proyecto tiene como objetivo el diseño, desarrollo e implementación de un sistema informático integral para la gestión, planificación y monitorización del entrenamiento de fuerza y la composición corporal. El proyecto se basa en una arquitectura cliente-servidor compuesta por una API RESTful desarrollada en Java con un framework llamado SpringBoot y una aplicación móvil nativa para el sistema operativo Android.

La motivación principal de este proyecto surge por una mayor concienciación sobre el cuidado físico y la ayuda de los principios científicos para lograr este objetivo, tales como la sobrecarga progresiva y el control de la fatiga. Normalmente la mayoría de usuarios de los gimnasios registra de manera ineficiente sus progresos en el gimnasio, utilizando notas de

papel, aplicaciones móviles genéricas no adaptadas o directamente no lo registran. Mi proyecto busca solucionar este problema ofreciendo una herramienta centralizada, rápida y precisa que permite al usuario registrar cada serie, repetición y carga movida de forma intuitiva.

En el mercado existen diversas herramientas comerciales ya consolidadas que abordan este nicho como Strong, Hevy o FitNotes. Aunque estas aplicaciones tienen una gran popularidad y ofrecen un alto grado de madurez, a menudo tienen interfaces sobrecargadas con funcionalidades secundarias, o restringen una parte de sus características como la analítica o el historial con modelos de suscripción económica.

Los aspectos más destacados y novedosos del proyecto radican en su enfoque funcional y técnico:

- ✓ **A nivel funcional:** se da prioridad a que la experiencia del usuario sea ágil y directa. Añadiendo variables avanzadas para controlar la auto regulación como el RPE (Escala de Esfuerzo Percibido) en el registro de series, un catálogo de rutinas globales y estandarizado y terminado con el seguimiento del peso y el porcentaje de grasa del usuario.
- ✓ **A nivel técnico:** el proyecto destaca por su rigor en el diseño de software. Se implementa un modelo de base de datos relacional (MySQL) normalizado con un enfoque moderno basado en Code-First y mapeo objeto-relacional (ORM Hibernate). También garantiza una alta escalabilidad, contraseñas encriptadas para la seguridad de los usuarios y una comunicación cliente-servidor altamente eficiente gracias a la librería Retrofit en el entorno Android.

Desarrollo

Revisión bibliográfica y fundamentación teórica

Para el desarrollo de GymProgress, se ha llevado a cabo un análisis previo del estado actual de las herramientas de software orientadas al registro del entrenamiento físico. La base teórica de este proyecto se sustenta en la creciente necesidad de aplicar principios científicos al entrenamiento de fuerza para alcanzar los objetivos deseados, prestando especial atención a conceptos clave como la sobrecarga progresiva y el control riguroso de la fatiga. A pesar de esta necesidad técnica, la observación del entorno revela que la mayoría de los usuarios de gimnasios registra sus progresos de manera ineficiente,

utilizando notas de papel, aplicaciones móviles genéricas no adaptadas, o simplemente omitiendo cualquier tipo de registro.

Al revisar las soluciones existentes en el mercado del software móvil, se constata que existen diversas herramientas comerciales ya consolidadas que abordan este nicho específico, entre las que destacan principalmente Strong, Hevy y FitNotes. El análisis de estas alternativas demuestra que gozan de una gran popularidad y ofrecen un alto grado de madurez funcional.

Sin embargo, esta revisión también ha puesto de manifiesto deficiencias y áreas de mejora significativas que justifican el desarrollo de una nueva propuesta. A pesar de su éxito, estas aplicaciones de la competencia a menudo presentan interfaces sobrecargadas con funcionalidades secundarias que entorpecen el registro rápido de datos. Adicionalmente, el acceso a características imprescindibles para el seguimiento a largo plazo, como la analítica avanzada o la visualización del historial completo, suele estar restringido mediante modelos de suscripción económica.

Frente a las alternativas comerciales analizadas, la fundamentación de GymProgress se basa en resolver estos problemas ofreciendo una herramienta centralizada, rápida y precisa. La propuesta se distingue por priorizar una experiencia de usuario ágil y directa a nivel funcional, integrando de forma accesible variables avanzadas para la autorregulación deportiva, como la Escala de Esfuerzo Percibido (RPE). De este modo, se plantea un sistema que descarta la sobrecarga de información y se centra exclusivamente en la eficiencia del usuario durante la sesión de entrenamiento.

Lenguajes empleados

Lenguajes de programación y marcado:

- ✓ **Java:** Usando como lenguaje principal para el desarrollo de la lógica del servidor (Backend) usando el framework Spring Boot. He elegido este lenguaje ya que es un lenguaje fuertemente tipado, robusto, orientado a objetos facilitando la implementación de arquitecturas seguras y escalables.
- ✓ **Kotlin:** Utilizado para el desarrollo nativo de la aplicación cliente (Frontend) en el sistema operativo Android. He optado por Kotlin ya que es el lenguaje oficial

recomendado por Google, además de su compatibilidad con Java, su sintaxis sencilla y la seguridad que ofrece contra errores de referencia nulas.

- ✓ **SQL:** Usado para la definición y la manipulación de datos en el sistema gestor de base de datos MySQL. Aunque su generación esta automatizada gracias a ORM Hibernate, comprenderlo es necesario para garantizar la integridad y normalización del modelo relacional.
- ✓ **XML y JSON:** Uso XML como lenguaje de marcado para el diseño de las interfaces de usuario y la configuración de manifiesto en Android. JSON se usa para las peticiones y respuestas HTTP entre la aplicación móvil y la API REST.

Entornos de Desarrollo Integrados (IDEs)

- ✓ **IntelliJ IDEA:** Usado para la programación de la API Backend. Ha sido desarrollado por JetBrains y es considerado el líder para Java y Spring Boot. Lo uso por sus altas capacidades de refactorización y depuración en tiempo real.
- ✓ **Android Studio:** Es el IDE oficial de Google usado para la construcción, testeo y empaqueta de la aplicación móvil. Se basa en la misma tecnología que IntelliJ y lo he escogido ya que proporciona herramientas exclusivas como el emulador de dispositivos virtuales, el gestor de dependencias Gradle y un editor visual de interfaces.

Despliegue y distribución del sistema

Ya que el proyecto tiene la arquitectura cliente-servidor, el proceso de despliegue y distribución se divide en dos: el alojamiento de la API (Backend) y la distribución de la aplicación (Frontend).

Despliegue del Backend (API REST y Base de Datos):

La lógica del servidor se compila y ejecuta usando Maven en un archivo ejecutable con formato .jar. Este tipo de empaquetado tiene la ventaja de incluir un servidor web Apache Tomcat embebido simplificando su despliegue.

Para que el servidor funcione necesita un Servidor Virtual Privado (VPS) o una Plataforma como Servicio (PaaS) orientada a la nube. El servidor deberá tener estas características:

- ✓ Disponer de un Entorno de Ejecución de Java.
- ✓ Tener la capacidad de habilitar y exponer puertos de red seguros para recibir y contestar las peticiones REST.
- ✓ Integrar o tener conexión en la misma red privada virtual con un Sistema Gestor de Bases de Datos Relacional (MySQL) para que la información sea persistente.

Distribución de la aplicación cliente (Android)

La distribución del cliente seguirá los estándares del sistema operativo móvil Android. El entorno Android Studio compila el código fuente en un paquete instalable.

Para las fases de prueba, depuración y evaluación del proyecto final la distribución se hará generando un archivo ejecutable APK que puede ser transferido e instalado directamente en cualquier dispositivo físico o emulador.

Para su posible comercialización y distribución el sistema será empaquetado en formato AAB que es el estándar optimizado y requerido por Google Play Store para que la publicación sea oficial, permitiendo que la tienda distribuya únicamente el código y los recursos necesarios.

Requisitos de los clientes

Para los clientes que quiera utilizar la aplicación GymProgress tengan una experiencia de usuario fluida tiene que cumplir con una serie de requisitos técnicos y funcionales:

- ✓ **Dispositivo de Hardware y Sistema Operativo:** el usuario debe usar un dispositivo móvil que posea el sistema operativo Android. Para que las librerías modernas usadas en el desarrollo de la interfaz y la gestión de red (como Retrofit) funcionen, el dispositivo debe contar como mínimo con la versión 8.0 Android.
- ✓ **Conectividad a internet:** como la aplicación sigue el modelo cliente-servidor es indispensable contar con una conexión a internet estable, puede ser a través de redes móviles o redes inalámbricas. Esto es necesario para el envío y recepción de los paquetes de datos JSON de la API RESTful alojada en el servidor para garantizar la persistencia y la sincronización de los entrenamientos.
- ✓ **Autenticación y Credenciales de Acceso:** para la privacidad de la información de salud y rendimiento de cada usuario, el acceso estará restringido. El cliente tiene que completar un registro inicial dando un correo electrónico válido y una contraseña. Luego se necesitarán estas credenciales para iniciar sesión en la aplicación y acceder a su perfil, donde se almacena sus rutinas, medias corporales y registros de series.
- ✓ **Permisos de la Aplicación:** el usuario debe conceder permisos básicos de acceso a la red durante la instalación o la primera vez que se ejecute la aplicación.

Licenciamiento

Todo el software desarrollado en el proyecto estará bajo los términos de la licencia MIT, un modelo de licencia de código abierto. Esta licencia permite el uso, la copia, modificación, fusión, publicación y distribución del software con la condición de que incluya una copia de la licencia original y el aviso de derechos de autor en todas las copias del código. Gracias al uso de esta licencia se fomenta la divulgación tecnológica en el ámbito académico y facilita posibles ampliaciones del proyecto.

En el proyecto he usado varias librerías, frameworks, y herramientas de terceros respetando los términos y condiciones de estos componentes:

- ✓ **Java y Spring Boot:** El framework utilizado para la API REST del servidor se distribuye bajo la licencia Apache License 2.0, una licencia permisiva que permite que las aplicaciones que lo usen sean de código abierto.
- ✓ **Hibernate ORM:** encargado del mapeo objeto-relacional tiene la licencia GNU Lesser General Public Licenser (LGPL v2.1). Esta licencia permite conectar dinámicamente la Librería de Hibernate en el proyecto.
- ✓ **MySQL:** se usa la versión MySQL community Server bajo la GNU GPLv2. Como es un servicio externo con el que se comunica nuestra API y no tiene código en la aplicación, se cumple con su licencia sin generar conflictos legales.
- ✓ **Android y Kotlin:** tanto Kotlin como el SDK de Android operan bajo la licencia Apache License 2.0.
- ✓ **Retrofit y Gson:** son librerías de terceros desarrolladas por la empresa Square Inc. Que se distribuyen bajo la Apache License 2.0.

Recursos

Hardware:

- ✓ **Para el desarrollo e implementación local:**
 - **CPU:** AMD Ryzen 7 7700X. Su arquitectura multinúcleo y su frecuencia de reloj permiten una compilación muy rápida, reduciendo los tiempos de construcción.
 - **Memoria RAM:** 32GB de memoria principal con una velocidad de 6000 MT/s. Este es uno de los componentes más importantes porque permite la ejecución de IntelliJ IDEA, Android Studio, el servidor Apache, MySQL y la máquina virtual de Android sin latencia.

- **Almacenamiento:** SSD M.2 con una capacidad de 2TB. Tiene tasas de lectura y escritura ultrarrápida para el manejo de muchos archivos pequeños de los repositorios ya para el arranque instantáneo de los servidores.
- **GPU:** NVIDIA GeForce RTX 4070. Se usa principal mente para la aceleración por hardware para renderizar de manera fluida la interfaz gráfica de los emuladores Android.
- ✓ **Para la Distribución:**
 - **Para el Backend:** la distribución del .jar y la base de datos requiere un Servidor Virtual Privado básico. Un servidor con 1 o 2 núcleos virtuales y 2GB de memoria RAM es suficiente para gestionar las peticiones HTTP.
 - **Para el Frontend:** el hardware requerido depende del cliente, lo único necesario es un dispositivo inteligente con Android 8.0 o superior.

Software

- ✓ **Software de Desarrollo e Implementación (Backend)**
 - **JDK:** He utilizado la versión 25 LTS de java como base para la compilación y ejecución del código del servidor.
 - **Spring Boot:** es el framework principal. Permite la creación de aplicaciones auto configurables y listas para la producción.
 - **Apache Maven:** se encarga de descargar automáticamente las librerías necesarias y compilar el proyecto.
 - **MySQL Community Server:** Sistema Gestor de Bases de Datos Relacionales.
- ✓ **Software de Desarrollo e Implementación (Frontend):**
 - **Android SDK:** es un conjunto de herramientas desarrollo que incluyen librerías de la API de Android, el compilador y un emulador de dispositivos para realizar pruebas.
 - **Gradle:** un sistema de automatización de construcción avanzado que usa Android Studio. Gestión librerías externas y empaqueta los recursos gráficos y el código fuente.
- ✓ **Herramientas Auxiliar y de Pruebas:**
 - **Postman:** la he usado para probar las APIs, simulando peticiones HTTP del cliente, verificar los códigos de estado de las respuestas y la estructura de los JSON devueltos antes de integrarlo en la aplicación móvil.

- **DBeaver:** una herramienta de administración de bases de datos universal. Usado para ver el esquema relacional, comprobar las tablas generadas por Hibernate y exportar diagramas Entidad-Relación.
- **Git y GitHub:** Git es un sistema de control de versiones y GitHub es una plataforma de alojamiento en la nube, ayudando a mantener un registro de los cambios efectos en el código.
- ✓ **Software para la Distribución:**
 - **Apache Tomcat:** Spring Boot incorpora un servidor Tomcat dentro del .jar permitiendo crear el servicio con una simple línea de comando.
 - **Android OS:** el software requerido del cliente es el sistema operativo Android, que interpreta y ejecuta el código.

Humanos

- ✓ **Fase de Análisis y Diseño (5 horas)**
- ✓ **Fase de Desarrollo del Servidor / Backend (15 horas)**
- ✓ **Fase de Desarrollo Cliente / Frontend (40 horas)**
- ✓ **Fase de Integración y Pruebas / Testing (10 horas)**
- ✓ **Fase de Documentación y Redacción de la memoria(5 horas)**

Previsión económica del coste del proyecto

Para establece el coste total de desarrollo de GymProgress, se ha realizado una estimación realista categorizando los gastos en tres partidas fundamentales: licencias de software, costes de infraestructura para el despliegue y costes de personal humano.

Costes de Software

Gran parte del desarrollo de la aplicación y el servidor se ha usado tecnologías de código abierto (Open Source) o licencias gratuitas para uso educativo y de desarrollo por lo que no supone un gasto económico.

- ✓ **Entornos de desarrollo:** Android Studio (Gratuito) e IntelliJ IDEA (Licencia Community). Coste: 0,00€
- ✓ **Gestión de Bases de Datos:** DBeaver y MySQL Community Server. Coste: 0,00€

Subtotal Software: 0,00€

Costes de infraestructura y Despliegue

Estos costes contemplan los servicios estrictamente necesarios para que el sistema esté operativo, alojado en la nube y disponible para su distribución a los usuarios finales. Se calcula una estimación para el primer año de vida del proyecto.

- ✓ **Servidor VPS (Backend y Base de Datos):** Servidor básico (1-2 núcleos, 2GB RAM) necesario para alojar la API Spring Boot y el sistema gestor MySQL. Estimación de 6 €/mes durante 12 meses. Coste: 72,00 €.
- ✓ **Licencia de Google Play Developer:** Pago único exigido por Google para habilitar la cuenta de desarrollador y poder publicar la aplicación Android en la tienda oficial. Coste: 25,00 €.

Subtotal Infraestructura: 97,00 €

3. Costes de Personal (Recursos Humanos)

Se ha estimado el coste de la mano de obra basándose en el salario medio de un desarrollador de software Junior, fijando un coste por hora de 18 €/h. Las horas invertidas se desglosan según las fases de ciclo de vida del proyecto:

- ✓ Fase de Análisis y Diseño (5 horas): 90,00 €
- ✓ Fase de Desarrollo del Servidor / Backend (15 horas): 270,00 €
- ✓ Fase de Desarrollo Cliente / Frontend (40 horas estimadas): 720,00 €
- ✓ Fase de Integración y Pruebas / Testing (10 horas estimadas): 180,00 €
- ✓ Fase de Documentación y Redacción de la Memoria (5 horas): 90,00 €

Subtotal Personal (75 horas totales): 1.350,00 €

Resumen del Presupuesto Total

Licencias de Software	0,00€
Infraestructura (Primer año)	97,00€
Personal (Desarrollo y Diseño)	1.350,00€
TOTAL	1447,00€

Descripción de la aplicación

Funcionamiento general

Descripción detallada del funcionamiento de la herramienta:

GymProgress es una aplicación móvil diseñada para ayudar a los usuarios a gestionar su entrenamiento físico. La aplicación permite construir una base de datos centralizada y estandarizada de ejercicios y rutinas predefinidas. A su vez tiene un entorno en tiempo real para que los usuarios puedan guiar sus sesiones de gimnasio con funciones como cronometrar, calcular la capacidad máxima de levantamiento (1RM) y registrando las series, repeticiones, pesos y esfuerzo percibido (RPE). Una vez terminado el entrenamiento la herramienta lo procesa, consolida un historial detallado y genera graficas del progreso físico a lo largo del tiempo.

Enumeración de los Roles

En la aplicación existen dos roles principales:

- ✓ **Usuario (Estándar):** Es el consumidor final de la herramienta. Su rol se centra en la ejecución de entrenamientos, el uso de las calculadoras integradas y el registro de su evolución corporal (peso y porcentaje de grasa).
- ✓ **Administrador:** su nivel de acceso es superior, aparte de poner todas las funciones de un usuario estándar también tiene acceso exclusivo a un panel de control desde el cual puede gestionar, crear y estructurar el contenido global (catálogo de ejercicios y rutinas) que luego usaran los usuarios.

Enumeración de Funcionalidades

Funcionalidades de Administrador:

- ✓ **Añadir Ejercicio Nuevo:** permite añadir un ejercicio al catálogo de ejercicios definiendo el nombre, grupo muscular, descripción y tipo de equipamiento.
- ✓ **Crear Rutina:** Crea un conjunto de ejercicios existentes, definiendo un nombre.

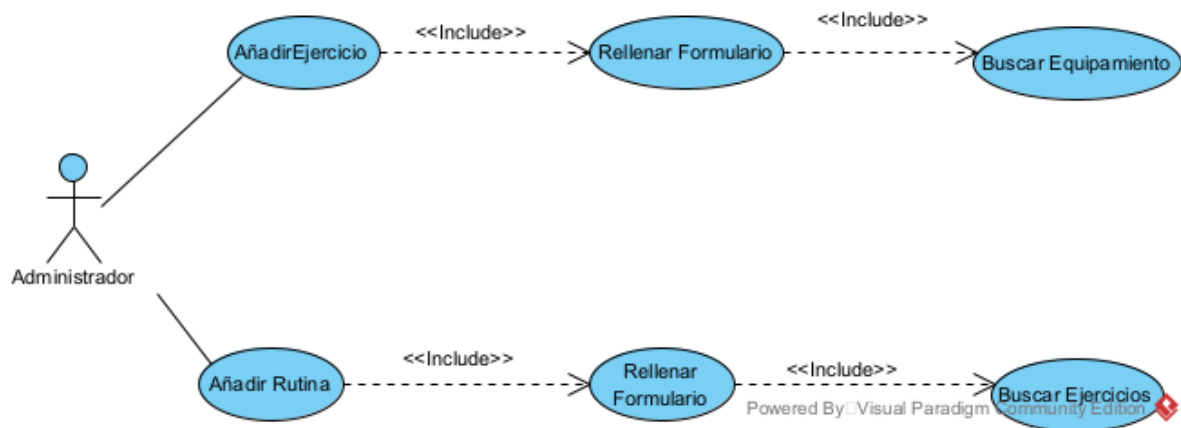
Funcionalidades de Usuario:

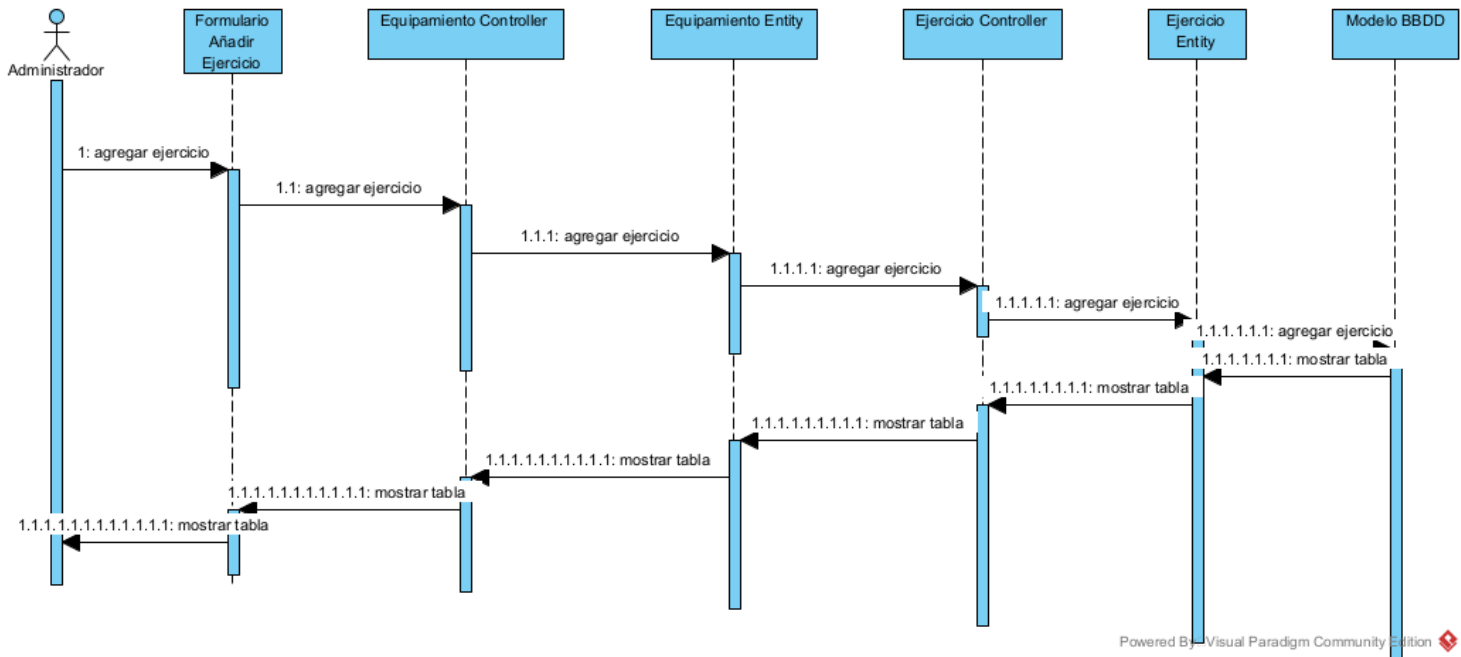
- ✓ **Entrenamiento libre:** inicia una sesión de entrenamiento completamente en blanco y va añadiendo ejercicio.

- ✓ Entrenar Rutina del Catálogo: selecciona una rutina predefinida que carga los ejercicios que se van a realizar en la sesión.
- ✓ Registro de series: función para llevar en tiempo real un registro de las repeticiones, el peso y el nivel de esfuerzo por cada serie, además de añadir notas.
- ✓ Herramientas en Vivo: como el cronómetro de la sesión o de descanso y la calculadora del 1 RM.
- ✓ Historial y Progreso: permite llevar un registro del peso corporal, porcentaje de grasa y visualización gráfica del progreso o consultar las sesiones pasadas por fecha.

Resultados Generados

- ✓ Catálogo





Arquitectura

Diseño de la base de datos

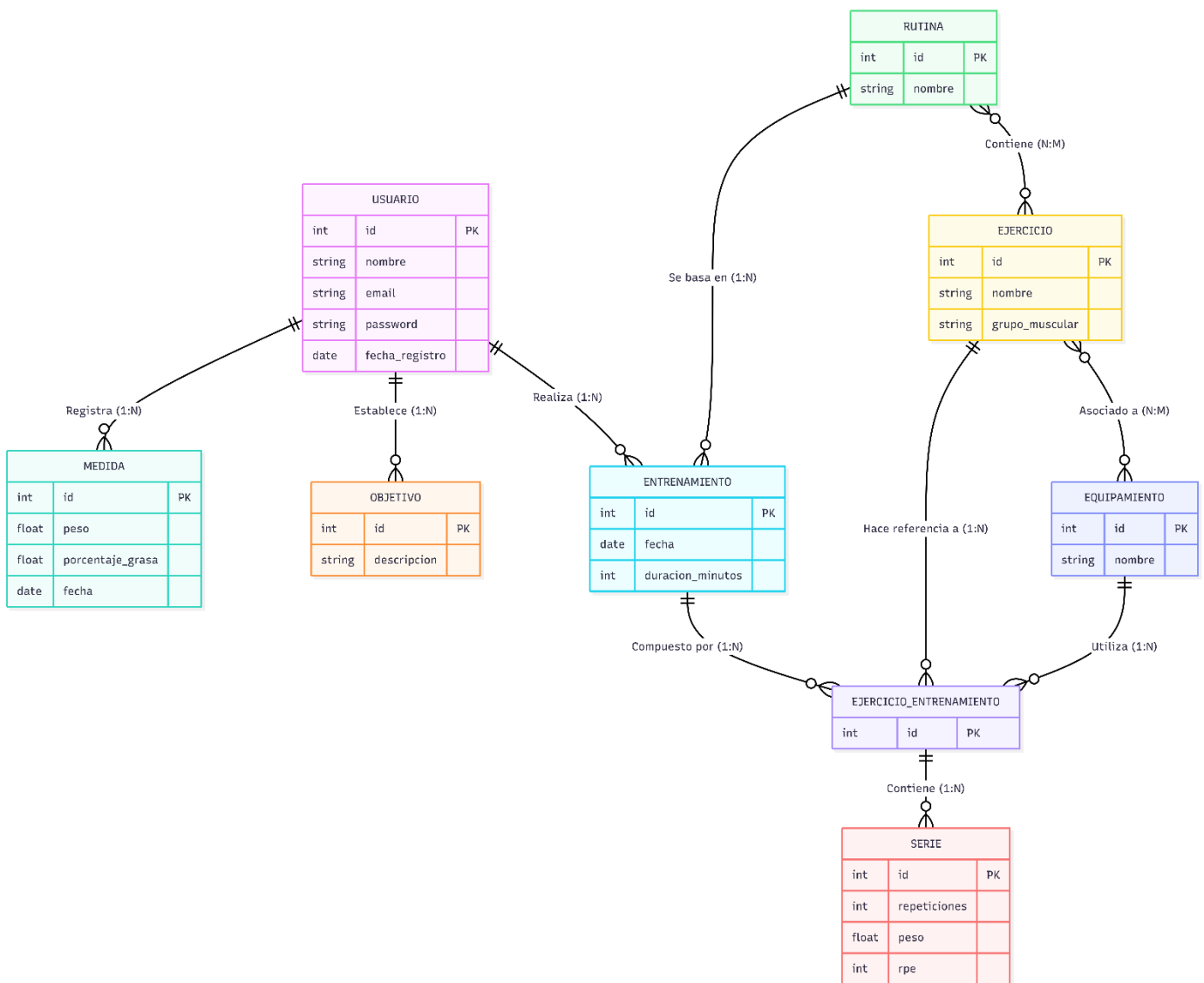
Modelo Entidad-Relación

- ✓ Un **Usuario** puede registrar múltiples **Medidas** a lo largo del tiempo y puede establecer múltiples **Objetivos**.
- ✓ Un **Usuario** puede realizar múltiples **Entrenamientos**.
- ✓ Un **Entrenamiento** puede basarse de forma opcional en un **Rutina**.
- ✓ Un **Entrenamiento** está compuesto por múltiples bloques de ejercicios realizados en esa sesión, representados por **Ejercicio_Entrenamiento**.
- ✓ Cada bloque de **Ejercicio_Entrenamiento** contiene a su vez múltiples **Series**.
- ✓ Cada bloque de **Ejercicio_Entrenamiento** hace referencia a un **Ejercicio** específico del catálogo y a un **Equipamiento** específico utilizado en esa sesión.
- ✓ Las **Rutinas** conforma un catálogo global y contienen múltiples **Ejercicios**.
- ✓ Los **Ejercicios** del catálogo global pueden estar asociados a múltiples tipos de **Equipamiento**.

Modelo Relacional

- ✓ **usuarios** (id_usuario, nombre, email, password, fecha_registro)
- ✓ **medidas** (id_medida, peso_corporal, porcentaje_grasa, fecha, id_usuario)
- ✓ **objetivos** (id, tipo, valor_objetivo, fecha_limite, completado, usuario_id)

- ✓ **equipamiento** (id, nombre)
- ✓ **ejercicios** (id_ejercicio, nombre, descripción, grupo_muscular)
- ✓ **rutinas** (id_rutina, nombre)
- ✓ **rutina_ejercicio** (id_rutina, id_ejercicio)
- ✓ **entrenamientos** (id_entrenamiento, fecha, duracion_minutos, id_usuario, id_rutina)
- ✓ **ejercicio_entrenamiento** (id_orden, notas, ejercicio_id, entrenamiento_id, equipamiento_id)
- ✓ **series** (id_serie, tipo, repeticioes, peso, rpe, ejercicio_entrenamiento_id)



Diccionario de Datos

Usuarios

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador único de usuario
Nombre	VARCHAR(255)	NOT NULL	Nombre completo o alias del usuario
Email	VARCHAR(255)	NOT NULL	Correo electrónico (credencial de acceso)
Password	VARCHAR(255)	NOT NULL	Contraseña encriptada por seguridad
Fecha_registro	DATE TIME	NOT NULL	Fecha y hora en la que se creó el registro

Medidas

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador único de medidas
Peso	DOUBLE	NOT NULL	Peso del usuario en kilogramos
Porcentaje_grasa	DOUBLE	Opcinal	Índice de grasa corporal del usuario
Fecha	DATE TIME	NOT NULL	Fecha exacta a de la toma de medidas
Usuario_id	BIGINT	FK	Referencia de usuario al que pertenece la medida

Objetivos

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto_increment	Identificador único de objetivo
Tipo	VARCHAR(255)	NOT NULL	Tipo de objetivo (ej, Peso, Grasa, Fuerza...)
valor_objetivo	DOUBLE	NOT NULL	Valor numérico a alcanzar
Fecha_limite	DATE	NULL	Fecha límite estimada para cumplir el objetivo
Completado	BIT(1)	NOT NULL	Indica si el objetivo se ha cumplido (0 o 1)
Usuario_id	BIGINT	FK, NULL	Referencia al usuario (usuarios.id)

Equipamiento

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador único de equipamiento
Nombre	VARCHAR(255)	UNIQUE, NOT NULL	Nombre del equipamiento

Ejercicios

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador único de usuario
Nombre	VARCHAR(255)	NOT NULL	Nombre del movimiento

Grupo_muscular	VARCHAR(255)	NOT NULL	Zona anatómica principal implicada
----------------	--------------	----------	------------------------------------

Ejercicio_equipamiento

Atributo	Tipo de dato SQL	Restricción	Descripción
Ejercicio_id	BIGINT	PK*, FK, NOT NULL	Referencia al ejercicio (ejercicios.id)
Equipamiento_id	BIGINT	PK*,FK, NOT NULL	Referencia al equipo compatible (equipamiento.id)

Rutinas

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador único de usuario
Nombre	VARCHAR(255)	NOT NULL	Nombre descriptivo

Rutina_ejercicio

Atributo	Tipo de dato SQL	Restricción	Descripción
Rutina_id	BIGINT	PK,FK	Referencia a la tabla rutinas
Ejercicio_id	BIGINT	PK, FK	Referencia a la tabla ejercicios

Entrenamientos

Atributo	Tipo de dato SQL	Restricción	Descripción
----------	------------------	-------------	-------------

Id	BIGINT	PK, Auto-incremental	Identificador único de usuario
Fecha	DATETIME	NOT NULL	Fecha y hora de realización
Duración_minutos	INT	NOT NULL	Tiempo total de duración de la sesión
Usuario_id	BIGINT	FK	Referencia al usuario que entrenó
Rutina_id	BIGINT	FK, Opcional	Referencia a la rutina base seguida (si la hay)

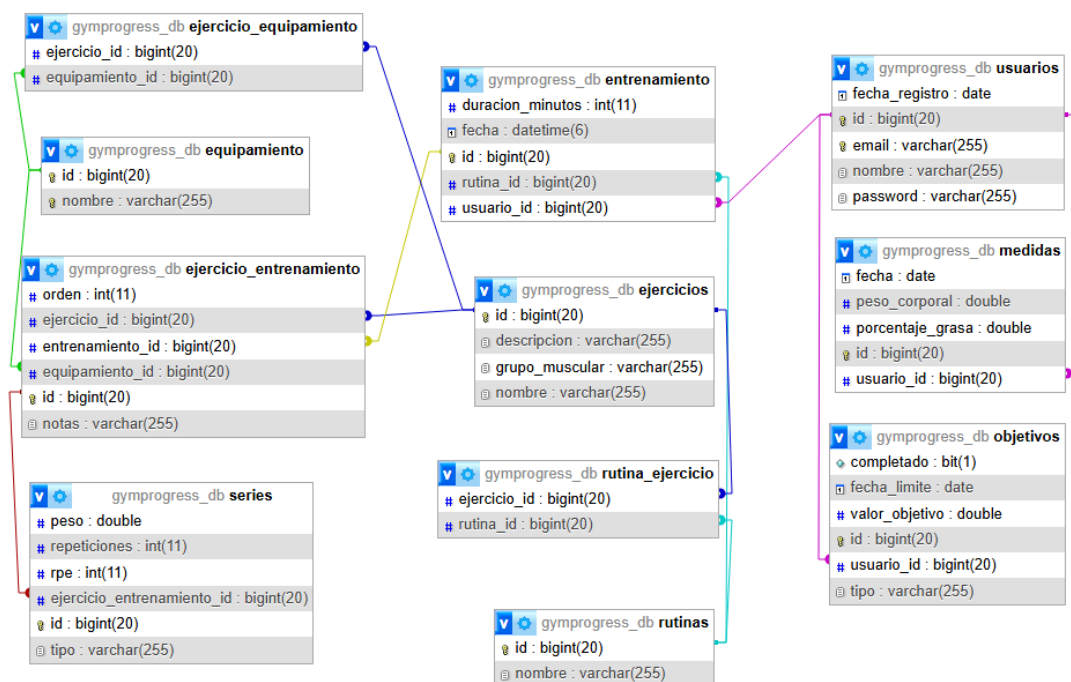
Ejercicio_entrenamiento

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador del bloque de ejercicio en la sesión.
Orden	INT(11)	NULL	Orden de la ejecución dentro de la sesión
Notas	VARCHAR(255)	NULL	Observaciones específicas para esta ejecución
Ejercicio_id	BIGINT	FK, NLL	Referencia al entrenamiento (ejercicios.id)
Entrenmaiento_id	BIGINT	FK, NULL	Referencia al entrenamiento (entrenamiento.id)
Equipamiento_id	BIGINT	FK, NULL	Equipo particular usado en la sesión (equipamiento.id)

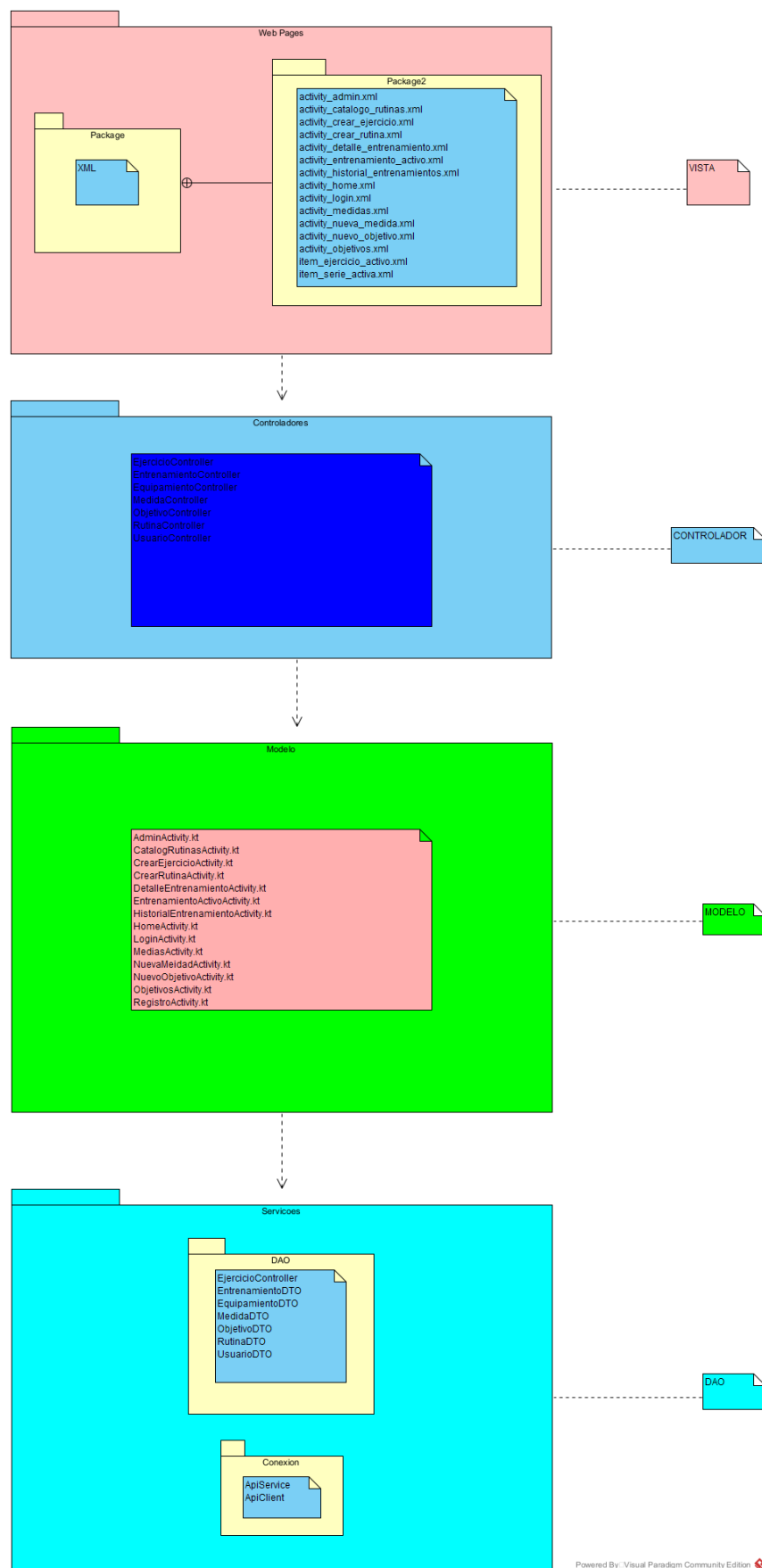
Series

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incrementa l	Identificador único de la serie realizada
Tipo	VARCHAR(255)	NULL	Tipo de serie (ej. Efectiva, Calentamiento)
Repeticiones	INT	NOT NULL	Número de alzamientos completados
Peso	DOUBLE	NOT NULL	Carga externa levantada en kilogramos
RPE	INT	NOT NULL	Escala de esfuerzo percibido (1-10)
Ejercicio_entrenamiento_id	BIGINT	FK, NULL	Vincula la serie con el bloque del ejercicio (ejercicio_entrenamiento.id)

Diagrama de clases



Arquitectura del sistema



Powered By: Visual Paradigm Community Edition

Interfaz

Características principales

Descripción de la interfaz y diseño general

La interfaz de la realizado priorizando el diseño minimalista, la usabilidad y la accesibilidad. Como la herramienta está enfocada para usarse durante entrenamientos en el gimnasio, este diseño evita la sobrecarga de información y utiliza una paleta de colores no agresiva.

Visualmente, el diseño se apoya fuertemente en las guías de Material Design, haciendo un gran uso de componentes como “MaterialCardView” para agrupar visualmente la información (por ejemplo, en las listas de ejercicios o en el historial de progreso). Una de las características clave es su adaptabilidad: la aplicación detecta automáticamente el tema del dispositivo del usuario y activa automáticamente el modo claro u oscuro, así garantiza siempre un contraste óptimo y una lectura cómoda de métricas importantes como los pesos y las repeticiones.

Navegación

La navegación es de la aplicación es directa y jerárquica, diseñada para que el usuario llegue donde necesita con los mínimos toques posibles.

- ✓ Todo empieza en una pantalla de Login/Registro clara y sin distracciones.
- ✓ Una vez autenticado, el usuario llega a la pantalla principal que actúa como centro de operaciones.
- ✓ Desde ahí, con botones amplios y accesibles (destacados en azul para las acciones principales y rojo para la salida), se ramifica en las distintas funcionalidades: entrenamientos, rutinas, medidas. La acción para ir atrás sigue el estándar nativo de Android, haciendo que el flujo sea completamente intuitivo.

Lenguajes empleados y validación

Todo el frontend de la aplicación se ha desarrollado de forma nativa para Android.

- ✓ **Lenguajes:** Para el diseño de las vistas (layouts) se ha utilizado XML a través de Android Studio, mientras que la lógica de la interfaz y la interacción con el usuario está programada con Kotlin.

- ✓ **Validación:** Para asegurar la integridad de los datos antes de ser enviados al servidor, la interfaz cuenta con validaciones en los propios formularios. Se comprueban que los campos obligatorios no estén vacíos, se valida el formato del email, y en los entrenamientos se comprueban que los inputs de las métricas contengan valores numéricos coherentes para evitar errores en la base de datos o en las gráficas de progreso.

Usabilidad/accesibilidad

Para el desarrollo de **GymProgress**, se ha tenido muy en cuenta que el entorno de uso principal es una sala de musculación. Esto significa que el usuario va a interactuar con la aplicación entre series, a menudo con fatiga física o prisas, por lo que la interfaz debe ser a prueba de frustraciones.

Mejoras de Usabilidad:

- ✓ **Diseño "Fatigue-Proof" (A prueba de fatiga):** Se han utilizado áreas de toque amplias, cumpliendo con el estándar mínimo de 48dp de Material Design. Botones críticos como "AÑADIR SERIE" o "TERMINAR Y GUARDAR" ocupan todo el ancho de la pantalla para que sea casi imposible fallar al pulsarlos, incluso sin mirar demasiado.
- ✓ **Teclados contextuales:** Para agilizar el registro de datos, los inputs están configurados para desplegar el teclado más eficiente en cada caso. Por ejemplo, al introducir los KG o las repeticiones se abre directamente el teclado numérico (`inputType="numberDecimal"`), evitando que el usuario tenga que cambiar manualmente desde el teclado alfanumérico.
- ✓ **Prevención de errores y feedback visual:** La aplicación bloquea acciones ilógicas para mantener la base de datos limpia. Por ejemplo, el campo RPE está restringido a valores del 1 al 10. Además, se utilizan componentes nativos como *Toasts* para avisar al usuario de que un entrenamiento se ha guardado correctamente o si le falta rellenar algún campo.

Características de Accesibilidad:

- ✓ **Contraste y Tema Dinámico:** Como se comentó en el apartado de diseño general, la app respeta el tema del sistema (Claro/Oscuro). El modo oscuro no solo ahorra batería en pantallas OLED, sino que reduce drásticamente la fatiga visual y evita

deslumbramientos en entornos cerrados, mejorando la accesibilidad para usuarios con fotofobia o sensibilidad ocular.

- ✓ **Textos escalables:** Todo el texto de las vistas XML está definido utilizando la unidad de medida sp (Scale-independent Pixels). Esto garantiza que, si un usuario tiene problemas de visión y configura la letra de su sistema operativo más grande, la fuente de GymProgress crecerá en proporción sin romper el *layout*.
- ✓ **Soporte para lectores de pantalla (TalkBack):** Se ha cuidado la semántica de la interfaz añadiendo el atributo `contentDescription` a los elementos visuales que no son texto puro (como el icono del "ojo" para mostrar/ocultar la contraseña en el login o los gráficos del progreso). De esta forma, los usuarios con discapacidad visual que utilicen TalkBack pueden navegar por la app enterándose perfectamente de qué hace cada botón.

Autoevaluación y conclusiones

Valoración del trabajo y dificultades encontradas

A nivel general, el desarrollo de **GymProgress** ha representado un reto técnico exigente, derivado principalmente de la decisión de abordar el proyecto de manera integral como *Full-Stack*. El objetivo no se limitó a la creación de una interfaz de usuario estética, sino a garantizar una operatividad real y robusta mediante la comunicación efectiva con un servidor centralizado.

Las principales dificultades encontradas durante el proceso de desarrollo fueron:

- ✓ **Curva de aprendizaje e integración:** La transición desde un desarrollo local orientado exclusivamente al dispositivo móvil hacia la estructuración de una API REST funcional supuso un desafío significativo. La implementación de la conexión entre el *frontend* en Android (Kotlin) y el *backend* desarrollado en Spring Boot requirió una inversión considerable de tiempo, especialmente en la gestión de peticiones asíncronas y el correcto *parsing* de las respuestas en formato JSON.
- ✓ **Seguridad y autenticación:** La implementación del sistema de seguridad mediante *JSON Web Tokens* (JWT) constituyó uno de los puntos críticos del proyecto. La configuración de los mecanismos para asegurar las rutas, la gestión precisa de la expiración de los *tokens* y la persistencia de la sesión del usuario en la aplicación

Android sin comprometer los estándares de seguridad, representaron los aspectos más complejos de la arquitectura del sistema.

Valoración de la herramienta o aplicación desarrollada

La aplicación cumple con el objetivo principal: registrar entrenamientos de fuerza de forma rápida y sin distracciones. La interfaz limpia basada en Material Design, la implementación del modo oscuro automático y el sistema de gráficas para visualizar la evolución del peso le dan un aspecto muy profesional. Además, contar con un Panel de Administrador deja la puerta abierta a gestionar el catálogo de ejercicios de forma centralizada, lo cual es un punto a favor muy grande de cara a su mantenimiento.

Conclusión final

- ✓ **Del diseño:** He aprendido que "menos es más". Al principio tiendes a querer meter mucha información en pantalla, pero centrarse en la usabilidad y usar componentes estándar (como los MaterialCardView) hace que la app sea mucho más intuitiva para el usuario final.
- ✓ **De la aplicación:** He conseguido montar una arquitectura sólida. Separar bien la lógica de presentación en Kotlin de la lógica de negocio y persistencia en el servidor Spring Boot + MySQL hace que la app sea muy escalable de cara al futuro.
- ✓ **Del tiempo empleado:** Como suele pasar en el desarrollo de software, la planificación inicial se queda corta. Se invierte mucho más tiempo del previsto en depurar errores, cuadrar los *endpoints* de la API y ajustar pequeños detalles de la interfaz para que se vea perfecta en diferentes tamaños de pantalla.
- ✓ **Conocimientos adquiridos:** Ha sido un salto cualitativo brutal. He pasado de ver tecnologías aisladas a entender el ciclo de vida completo de un producto de software: desde el diseño de la base de datos y la creación de la API con Java, hasta el consumo de esos datos y la creación de la interfaz nativa en Android.

Líneas de investigación futuras

- ✓ **Expansión del Ecosistema y Multiplataforma:** Actualmente, el cliente (Frontend) está desarrollado de forma nativa exclusivamente para el sistema operativo Android utilizando Kotlin. Una línea de trabajo sería el desarrollo de una versión

para el ecosistema iOS, ya sea de forma nativa o mediante tecnologías multiplataforma.

- ✓ **Funcionalidades Sociales y de Comunidad:** Se podría investigar la implementación de mecánicas sociales que permitan a los usuarios estandarizar y compartir sus rutinas predefinidas con la comunidad, o incluso permitir que entrenadores profesionales distribuyan planes de entrenamiento directamente a través de la infraestructura de GymProgress.

Bibliografía

- ✓ Apache Software Foundation. (s. f.). *Apache Maven project*. <https://maven.apache.org/>
- ✓ DBeaver Community. (s. f.). *DBeaver documentation*. <https://dbeaver.com/docs/wiki/>
- ✓ Git. (s. f.). *Git documentation*. <https://git-scm.com/doc>
- ✓ Google Developers. (s. f.). *Documentación oficial para desarrolladores de Android*. <https://developer.android.com/docs>
- ✓ Google Design. (s. f.). *Material design guidelines*. <https://m3.material.io/>
- ✓ Gradle Inc. (s. f.). *Gradle user manual*. <https://docs.gradle.org/>
- ✓ IETF (Internet Engineering Task Force). (2015). *RFC 7519: JSON Web Token (JWT)*. <https://datatracker.ietf.org/doc/html/rfc7519>
- ✓ JetBrains. (s. f.). *Kotlin programming language documentation*. <https://kotlinlang.org/docs/home.html>
- ✓ Oracle. (s. f.). *Java SE documentation* (Versión 25 LTS). <https://docs.oracle.com/en/java/>
- ✓ Oracle. (s. f.). *MySQL Community Server documentation*. <https://dev.mysql.com/doc/>
- ✓ Postman. (s. f.). *Postman learning center*. <https://learning.postman.com/>
- ✓ Red Hat. (s. f.). *Hibernate ORM documentation*. <https://hibernate.org/orm/documentation/>
- ✓ Spring IO. (s. f.). *Spring Boot reference documentation*. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- ✓ Square, Inc. (s. f.). *Gson user guide*. <https://github.com/google/gson>

- ✓ Square, Inc. (s. f.). *Retrofit: A type-safe HTTP client for Android and Java*.
<https://square.github.io/retrofit/>

Manual de Usuario



Pantalla de Inicio de Sesión (Login)

Descripción general y uso:

Es la prima pantalla con la que interactúa el usuario al iniciar la aplicación. Su objetivo principal es permitir el acceso seguro a los usuarios que ya posean una cuenta, autenticando sus credenciales para acceder a sus rutinas y progresos personales.

Elementos de la interfaz y uso:

- ✓ **Campo “Correo electrónico”:** Área de texto donde el usuario debe introducir la dirección de email asociada a su cuenta.
- ✓ **Campo “Contraseña”:** Área de texto para introducir la clave de seguridad
 - Funcionalidad de visibilidad: A la derecha de este campo se encuentra un icono con forma de ojo. Al pulsar sobre él, el usuario puede alternar entre ocultar y mostrar la contraseña introducida, facilitando la corrección de errores al escribir.
- ✓ **Botón “ENTRAR”:** Botón de acción principal. Tras introducir las credenciales, el usuario debe pulsar este botón para que el sistema valide los datos y le dé acceso a la pantalla principal de la aplicación.
- ✓ **Enlace de registro (“No tienes cuenta? Regístrate aquí”):** Texto interactivo situado en la parte inferior. Si un nuevo usuario descarga la aplicación y no tiene credenciales previas, al pulsar sobre este enlace será redirigido a la pantalla de creación de cuentas.

Pantalla de Registro (Crear cuenta)

Descripción general:

Esta pantalla permite a los nuevos usuarios darse de alta en la plataforma GymProgress. Su función es recopilar la información básica necesaria para crear un perfil personal único, donde el usuario podrá guardar de forma segura sus rutinas, medidas y la evolución de sus entrenamientos.

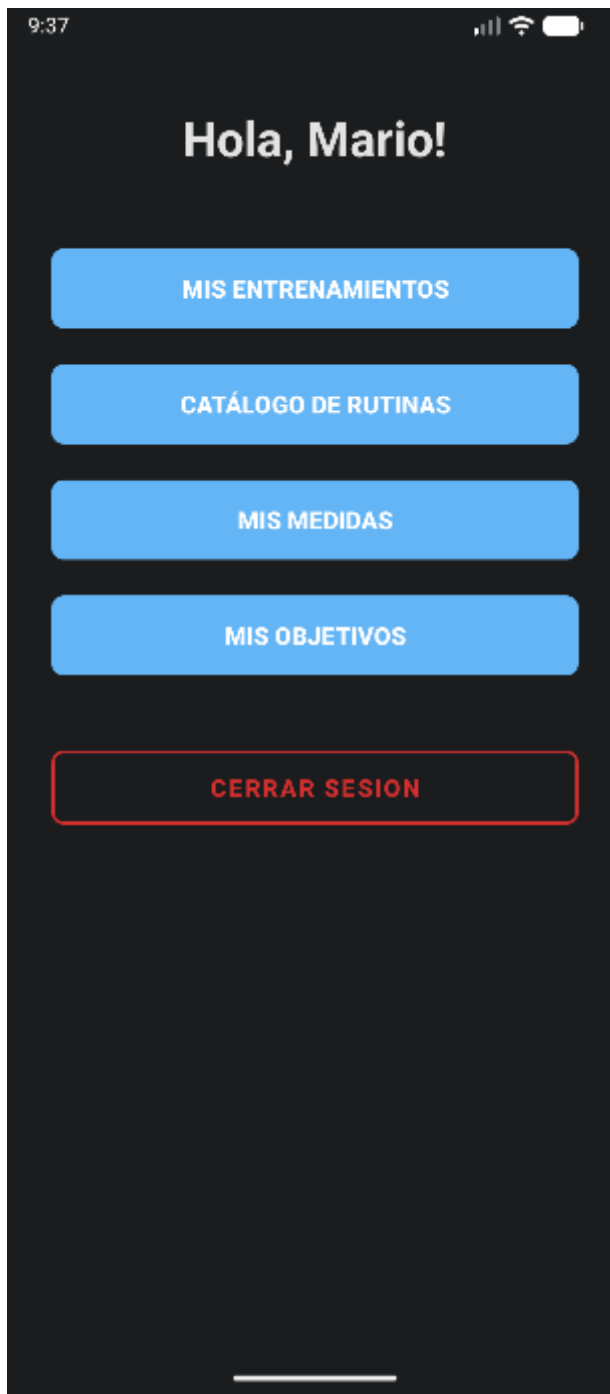
Elementos de la interfaz y uso:

- ✓ **Campo “Nombre Completo”:**
Área de texto donde el usuario debe introducir su nombre y apellidos. Este dato se utilizará para dirigirse al usuario dentro de la aplicación, ofreciendo una experiencia más personalizada.
- ✓ **Campo “Correo electrónico”:**
Área de texto destinada a registrar la dirección de email del usuario. Este correo funcionará como identificador único para iniciar sesión en el futuro y como vía de contacto o recuperación de cuenta si fuera necesario.
- ✓ **Campo “Contraseña”:** Área de texto para establecer la clave de seguridad de la nueva cuenta.
 - Funcionalidad de visibilidad: Al igual que en la pantalla de inicio de sesión, incorpora un icono en forma de ojo en el margen derecho. Al pulsarlo, el usuario puede revelar los caracteres introducidos para asegurarse de que ha escrito la contraseña correctamente antes de crear la cuenta.
- ✓ **Botón “REGISTRARSE”:** Botón de acción principal. Una vez cumplimentados todos los campos correctamente, el usuario debe pulsar este botón. El sistema procesará la

solicitud, creará el nuevo perfil en la base de datos y dará acceso al interior de la aplicación.

- ✓ **Enlace “Ya tengo una cuenta. Iniciar sesión”:** Texto interactivo de navegación situado en la parte inferior. Actúa como vía de escape en caso de que el usuario haya llegado a esta pantalla por error; al pulsarlo, el sistema lo devuelve inmediatamente a la pantalla de Inicio de Sesión.

Pantalla Principal (Menú de navegación)



Descripción general:

Es la pantalla principal, a la cual el usuario accede inmediatamente después de iniciar sesión con éxito. Funciona como el centro de control de la aplicación, ofreciendo una navegación clara y directa hacia todas las herramientas de gestión de entrenamiento y seguimiento físico.

Elementos de la interfaz y uso:

- ✓ **Mensaje de bienvenida (“Hola, [Nombre]!”):** Encabezado dinámico en la parte superior que saluda al usuario utilizando el nombre proporcionado durante su registro. Esto aporta un trato personalizado a la experiencia de uso.
- ✓ **Botón “Mis ENTRENAMIENTOS”:** Al pulsar este botón, el usuario accede a la sección donde puede gestionar, visualizar o registrar sus sesiones de entrenamiento personales y su historial de actividad.
- ✓ **Botón “CATÁLOGO DE RUTINAS”:** Dirige al usuario a una biblioteca o listado de rutinas de ejercicio predefinidas, permitiéndole explorar y seleccionar nuevos planes de entrenamiento.

- ✓ **Botón "MIS MEDIDAS"**: Da acceso al módulo de seguimiento físico. Aquí el usuario puede registrar, actualizar y consultar su evolución corporal (peso, perímetros musculares, etc.) a lo largo del tiempo.
- ✓ **Botón "MIS OBJETIVOS"**: Permite al usuario entrar a la sección dedicada a la fijación y seguimiento de metas a corto o largo plazo, ayudando a mantener la motivación en sus entrenamientos.
- ✓ **Botón "CERRAR SESION"**: Ubicado en la parte inferior y destacado con un estilo visual diferente (bordes y texto en rojo) para prevenir pulsaciones accidentales. Al accionarlo, se cierra la sesión actual del usuario por motivos de seguridad y se le redirige de vuelta a la pantalla de Inicio de Sesión.

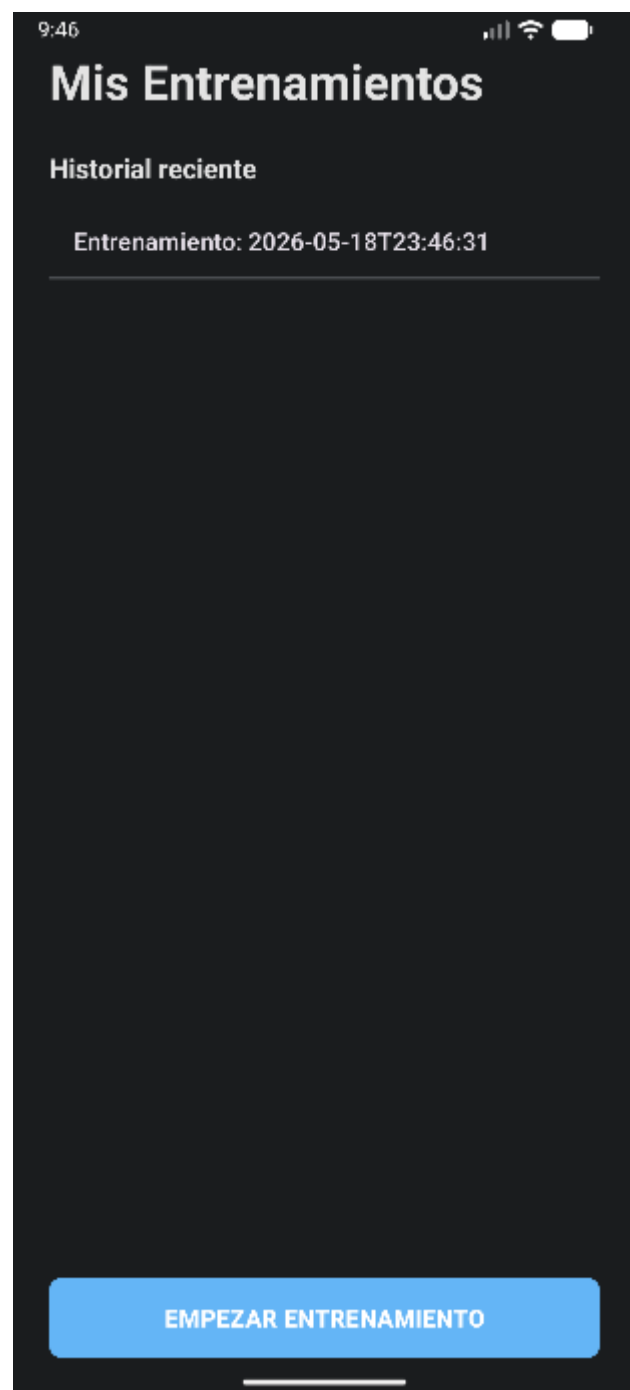
Pantalla de Mis Entrenamientos

Descripción general:

Esta interfaz actúa como el registro histórico y el punto de partida para la actividad física del usuario. Permite visualizar de un vistazo rápido las sesiones de ejercicio realizadas previamente y ofrece el acceso directo para comenzar una nueva rutina de entrenamiento.

Elementos de la interfaz y uso:

- ✓ **Sección "Historial reciente"**: Área que muestra una lista cronológica de las sesiones de entrenamiento guardadas por el usuario.
 - *Registros de entrenamiento*: Cada elemento de la lista representa una sesión completada, identificada mediante la etiqueta "Entrenamiento" acompañada de la fecha y la hora exacta en la que se registró (por ejemplo,



2026-05-18). Esta lista permite al usuario llevar un control temporal de su constancia.

- ✓ **Botón "EMPEZAR ENTRENAMIENTO"**: Es el botón de acción principal, destacado en la parte inferior de la pantalla. Al pulsarlo, el usuario iniciará un nuevo flujo de trabajo para registrar una sesión de ejercicio en tiempo real o introducir los datos de su rutina actual.

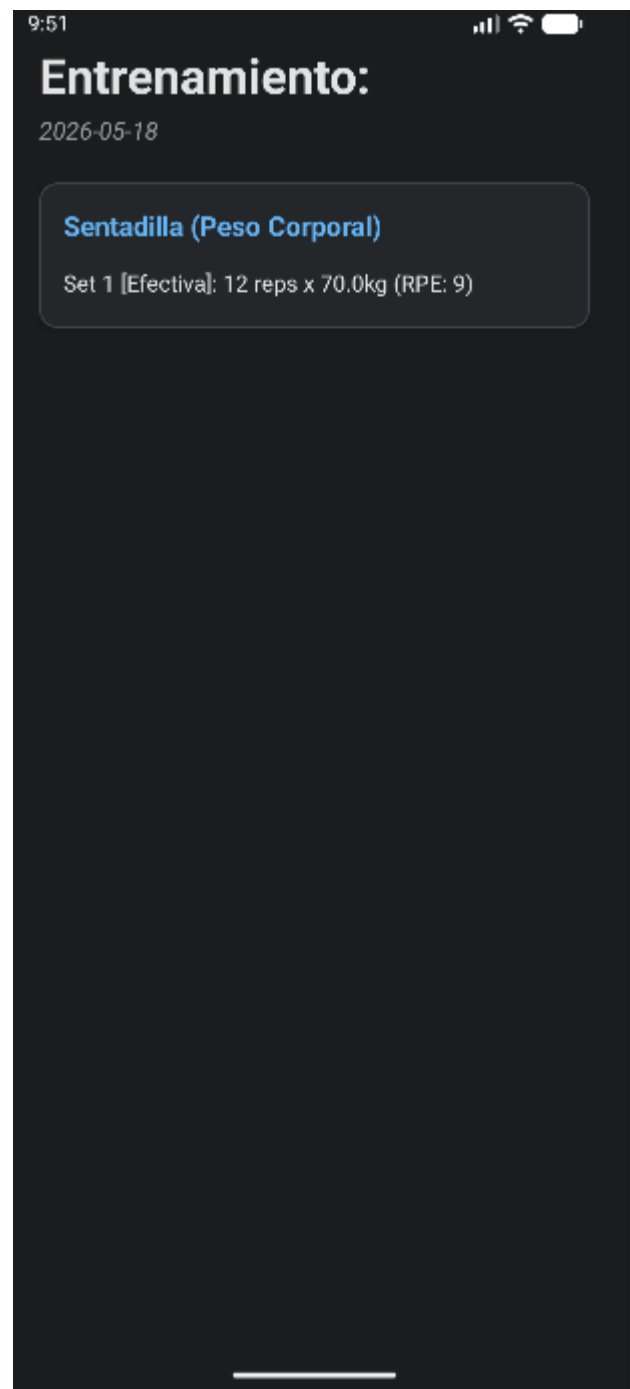
Detalle de la Sesión de Entrenamiento

Descripción general:

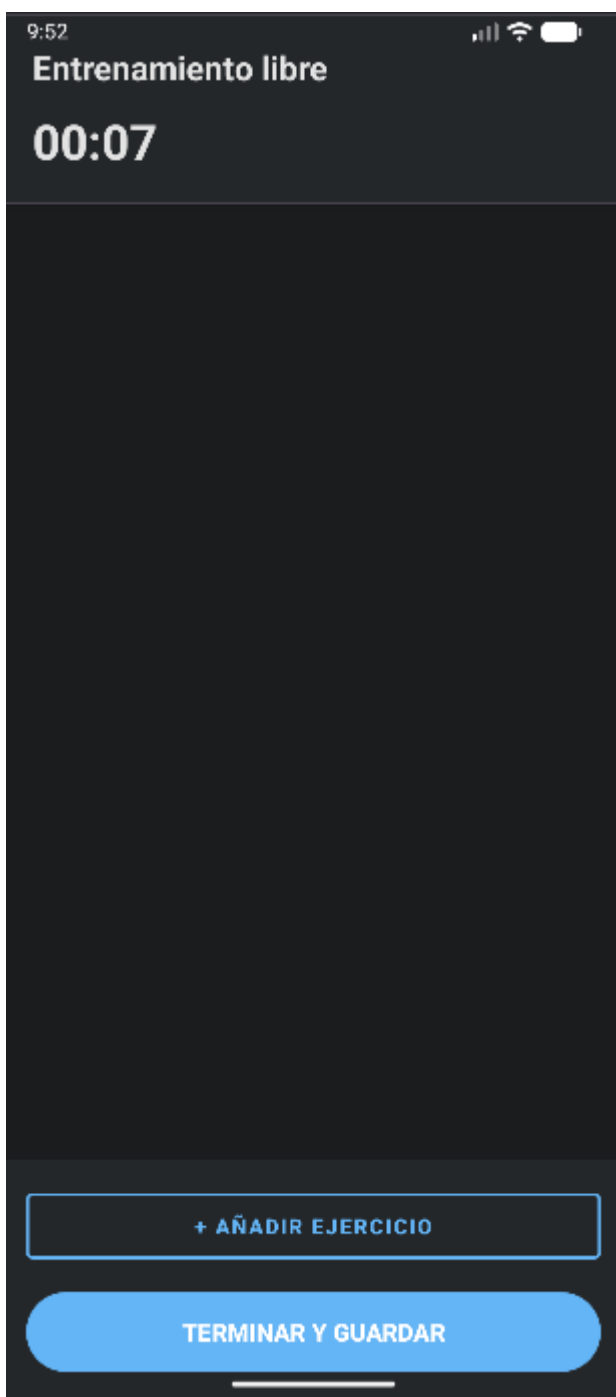
Esta pantalla ofrece una vista pormenorizada de una sesión de entrenamiento específica. Su objetivo es que el usuario pueda consultar de forma estructurada qué ejercicios realizó en una fecha determinada, así como el volumen de trabajo y la intensidad de cada una de las series completadas.

Elementos de la interfaz y uso:

- ✓ **Encabezado principal**: Muestra el título "Entrenamiento:" seguido de la fecha en la que se registró la sesión (por ejemplo, 2026-05-18), sirviendo como confirmación visual del día que se está consultando.
- ✓ **Tarjeta de Ejercicio (Card)**: Cada ejercicio realizado durante la sesión se representa mediante un bloque visual independiente.
 - **Nombre del ejercicio**: Título resaltado en color azul (ej. "Sentadilla (Peso Corporal)") que indica el movimiento ejecutado.



- **Desglose de la serie (Set):** Debajo del título, se lista detalladamente cada ronda de trabajo realizada para ese ejercicio. La información incluye:
 - **Número de serie y tipo:** Indica el orden de la serie (ej. "Set 1") y su clasificación (ej. "[Efectiva]", lo que denota que es una serie de trabajo real y no de calentamiento).
 - **Volumen y carga:** Muestra las repeticiones ejecutadas ("12 reps") multiplicadas por el peso movido ("70.0kg").
 - **Intensidad percibida (RPE):** Muestra la Escala de Esfuerzo Percibido (ej. "RPE: 9"), indicando al usuario lo cerca que estuvo del fallo muscular en esa serie concreta.



Flujo de Entrenamiento Libre (En curso)

Descripción general:

Este conjunto de interfaces conforma el entorno de trabajo activo donde el usuario registra su entrenamiento en tiempo real. El sistema está diseñado para ser ágil e intuitivo, minimizando las distracciones durante la sesión de ejercicio mientras permite registrar una gran cantidad de variables de rendimiento.

Pantalla principal y Cronómetro Global

- ✓ **Temporizador general:** Situado en la parte superior izquierda (ej. "00:07"), este cronómetro se inicia automáticamente al comenzar la sesión. Registra el tiempo total transcurrido del entrenamiento, permitiendo al usuario controlar la duración total de su rutina.
- ✓ **Botones de acción inferior: * "+ AÑADIR EJERCICIO":** Botón secundario que despliega el catálogo para incorporar nuevos movimientos a la sesión actual.

- **"TERMINAR Y GUARDAR"**: Botón principal que finaliza la sesión activa y guarda todos los datos registrados en el historial del usuario.

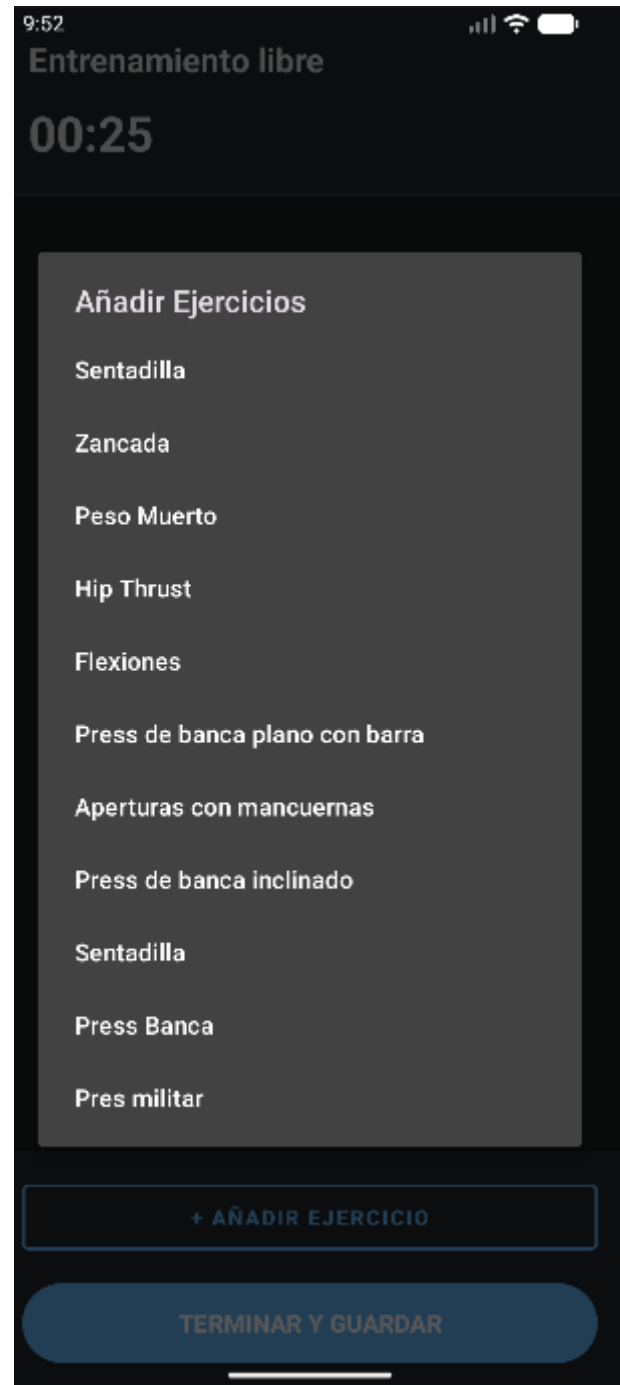
Selección de Ejercicios

- ✓ Al pulsar en "+ AÑADIR EJERCICIO", se despliega un menú inferior (*Bottom Sheet*) titulado "Añadir Ejercicios".
- ✓ Este menú contiene una lista desplazable con todos los ejercicios disponibles en la base de datos de la aplicación (ej. Sentadilla, Zancada, Peso Muerto, etc.). El usuario solo debe pulsar sobre el nombre del ejercicio deseado para que este se añada a su pantalla de entrenamiento activo.

Registro de Series y Variables (Tarjeta de Ejercicio)

Una vez seleccionado un ejercicio (ej. "Hip Thrust"), se genera una tarjeta de registro con los siguientes elementos:

- ✓ **Desplegable de Equipamiento:** Permite especificar el material utilizado (ej. barra, mancuernas, máquina).





- ✓ **Campo de Notas:** Área de texto opcional para que el usuario apunte sensaciones, recordatorios de técnica o ajustes específicos para ese ejercicio (ej. "Rango de movimiento completo").
- ✓ **Métricas de la Serie:** Campos numéricos para introducir el rendimiento exacto de cada ronda:
 - **REPS:** Número de repeticiones completadas.
 - **KG:** Peso utilizado en kilogramos.
 - **RPE (1-10):** Escala de Esfuerzo Percibido, que evalúa la intensidad de la serie.
- ✓ **Botón "+ AÑADIR SERIE":** Confirma y guarda los datos introducidos en los campos de métricas, registrando esa ronda de trabajo como completada.

Temporizador de Recuperación

- ✓ Tras registrar una serie, el sistema asiste al usuario mostrando una ventana emergente (*Dialog*) de "**TIEMPO DE RECUPERACIÓN**".
- ✓ Muestra un cronómetro en cuenta regresiva (destacado en color rojo) para controlar los tiempos de descanso entre series, optimizando así el rendimiento.
- ✓ **Botón "VOLVER AL LÍO":** Permite al usuario saltarse el tiempo de descanso restante y cerrar la ventana emergente manualmente si decide reanudar el ejercicio antes de que el cronómetro llegue a cero.



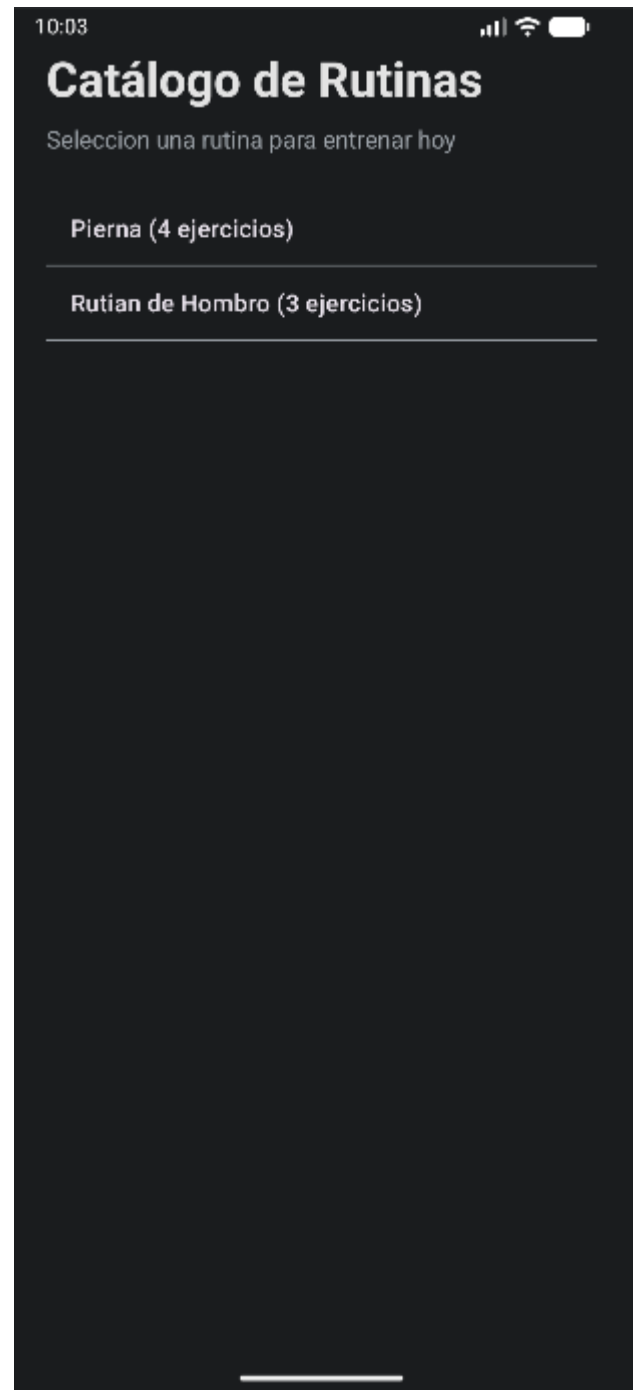
Catálogo de Rutinas

Descripción general:

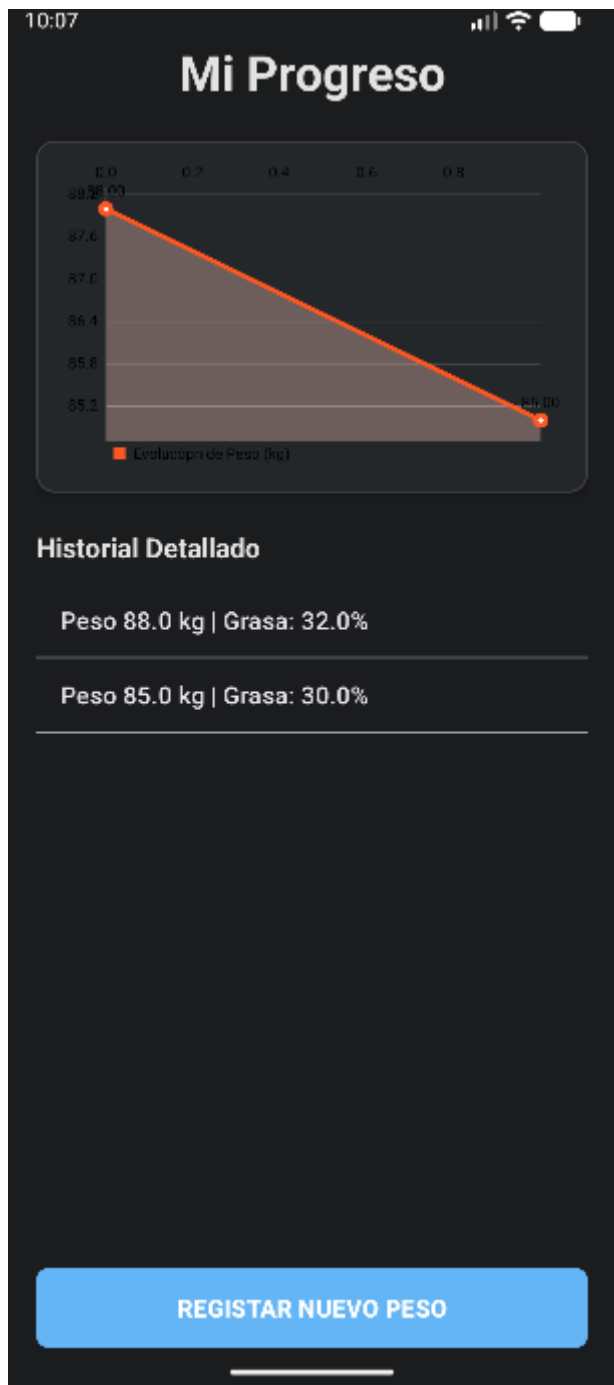
Esta pantalla actúa como una biblioteca de entrenamientos predefinidos. Su objetivo principal es ofrecer al usuario la comodidad de seleccionar plantillas de entrenamiento ya estructuradas, agilizando el inicio de la sesión en lugar de tener que añadir los ejercicios uno a uno de forma manual (como ocurre en el modo de "Entrenamiento libre").

Elementos de la interfaz y uso:

- ✓ **Texto instructivo:** Debajo del título principal, la aplicación presenta una breve indicación ("Selecciona una rutina para entrenar hoy") que guía al usuario de forma clara sobre la acción esperada en esta vista.
- ✓ **Lista de Rutinas Guardadas:** Área central que despliega, en formato de lista interactiva, todas las plantillas de entrenamiento disponibles en el perfil del usuario.
 - **Nombre de la rutina:** Cada elemento de la lista muestra un título principal (ej. "Pierna") que identifica el grupo muscular objetivo o el propósito específico del entrenamiento.
 - **Resumen de volumen:** Junto al nombre, se incluye un indicador entre paréntesis con la cantidad de ejercicios que componen dicha rutina (ej. "4 ejercicios"). Esto proporciona al usuario información útil de un solo vistazo, ayudándole a decidir qué rutina realizar en función del tiempo que tenga disponible.



- **Selección e inicio:** Al pulsar sobre cualquiera de las rutinas listadas, la



aplicación cargará automáticamente esa plantilla, dirigiendo al usuario a la pantalla de entrenamiento activo con los ejercicios ya precargados.

Pantalla de Seguimiento (Mi Progreso)

Descripción general:

Esta sección está dedicada a la monitorización de la evolución física del usuario. Proporciona un entorno visual y analítico donde se puede evaluar el impacto de los entrenamientos a lo largo del tiempo, consolidando los datos antropométricos registrados en información clara y comprensible.

Elementos de la interfaz y uso:

- ✓ **Gráfico de evolución ("Evolución de Peso"):** Representación visual mediante un gráfico de líneas que muestra la tendencia del peso corporal del usuario. Permite identificar rápidamente de un solo vistazo la trayectoria (pérdida, ganancia o

mantenimiento) sin necesidad de analizar los datos numéricos crudos.

- ✓ **Sección "Historial Detallado":** Área que lista cronológicamente las mediciones exactas introducidas en el sistema.
 - Cada registro (ítem de la lista) desglosa los valores específicos de esa toma de datos, mostrando métricas clave como el peso total (en kg) y el porcentaje de

grasa corporal, ofreciendo así un contexto más profundo que el simple peso en báscula.

- ✓ **Botón "REGISTRAR NUEVO PESO"**: Botón de acción principal ubicado en la zona inferior de la pantalla. Al pulsarlo, el usuario activa el flujo para introducir una nueva entrada con sus medidas actuales, actualizando automáticamente tanto el historial detallado como el gráfico de evolución.

Pantalla de Nueva Medida

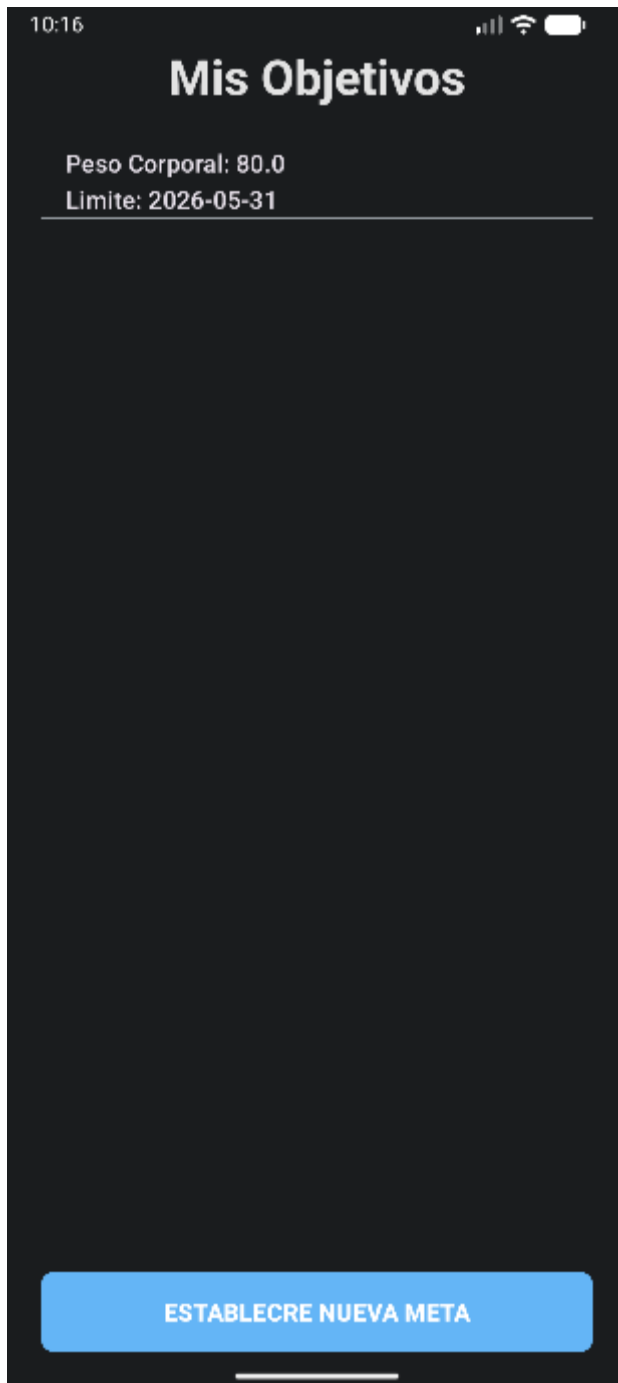
Descripción general:

Esta interfaz proporciona un formulario claro y conciso para que el usuario introduzca sus datos antropométricos más recientes. Su diseño minimalista busca facilitar una entrada de datos rápida y sin fricciones, alimentando directamente el historial y los gráficos de la sección de progreso.

Elementos de la interfaz y uso:

- ✓ **Campo "Peso actual"**: Área de entrada de datos donde el usuario debe teclear su peso corporal en ese momento.
- ✓ **Campo "Porcentaje de Grasa"**: Área de entrada complementaria para registrar el índice de grasa corporal, permitiendo un seguimiento más exhaustivo de la composición corporal más allá del peso en báscula.
- ✓ **Botón "GUARDAR"**: Botón de acción principal. Al accionarlo, el sistema valida la información introducida, la almacena de forma segura en la base de datos asociada al

perfil del usuario y redirige automáticamente a la pantalla de "Mi Progreso", donde los



nuevos datos ya aparecerán reflejados en el historial y en la gráfica de evolución.

Pantalla de Obejivos

Descripción general:

Esta vista muestra el listado de los objetivos que el usuario tiene actualmente en curso. Su finalidad es servir como un recordatorio visual rápido de las metas fijadas (por ejemplo, alcanzar un peso determinado) y los plazos establecidos para lograrlas, ayudando a mantener el enfoque y la motivación.

Elementos de la interfaz y uso:

- ✓ **Registro de Objetivo:** Cada meta activa se visualiza como un bloque de información en forma de lista.
 - **Métrica y valor objetivo:** La primera línea indica la variable física a mejorar y la cifra exacta que se desea alcanzar (ej. "Peso Corporal: 80.0").
 - **Fecha límite (Límite):** La segunda línea establece el horizonte temporal o plazo máximo que el usuario se ha marcado para cumplir dicho objetivo (ej. "2026-05-31").
- ✓ **Botón "ESTABLECER NUEVA META":** Este botón permanece fijo en la parte inferior de la pantalla, permitiendo al usuario seguir añadiendo múltiples objetivos simultáneos a su lista.

Pantalla de Nueva Meta

Descripción general:

Esta interfaz presenta el formulario interactivo mediante el cual el usuario puede definir y configurar un nuevo objetivo de entrenamiento o composición corporal. Está diseñada para ser flexible, permitiendo registrar desde metas de peso hasta marcas específicas en ejercicios concretos, asociándolas a un horizonte temporal determinado.

Elementos de la interfaz y uso:

- ✓ **Menú desplegable "¿Qué quieres conseguir?":** Permite al usuario seleccionar la categoría o métrica principal de su objetivo. Al pulsar sobre él, se despliega una lista de opciones predefinidas (como "Peso Corporal", "Porcentaje de Grasa", etc.).
- ✓ **Campo "Valor":** Área de texto donde el usuario debe introducir la cifra exacta que desea alcanzar.
 - *Texto de ayuda (Hint):* El campo incluye ejemplos transparentes ("ej.75kg, 12%grasa, 100kg banca") para guiar al usuario sobre el formato de los datos que debe introducir.
- ✓ **Sección "Fecha límite":**

- **Botón "SELECCIONAR FECHA":** Al pulsarlo, la aplicación despliega un calendario interactivo nativo del dispositivo, permitiendo al usuario escoger el día exacto, mes y año límite para cumplir su meta.
- **Indicador de estado:** Justo debajo del botón, un texto dinámico (que inicialmente muestra "No seleccionada") se actualizará automáticamente con la fecha escogida por el usuario tras usar el calendario.
- ✓ **Botón "GUARDAR OBJETIVO":** Es la llamada a la acción principal. Al pulsarlo, el sistema valida que todos los campos estén cumplimentados. Si la información es correcta, el objetivo se registra en el perfil del usuario y la aplicación lo redirige de vuelta a la lista de "Mis Objetivos", donde aparecerá la nueva meta.

Plan de negocio

Áreas de interés, sector y mercado dirigido

El proyecto se enfoca en el sector tecnológico orientado a la salud y bienestar, más concretamente se dirige al mercado de las aplicaciones móviles de productividad y registro deportivo.

El área de interés principal es el entrenamiento de fuerza (musculación, powerlifting, hipertrofia), un nicho de mercado en constante crecimiento donde los usuarios buscan digitalizar el clásico “cuaderno de entrenamiento” para aplicar el principio de sobrecargar progresiva basándose en datos reales.

Caracterizar el producto o servicio a ofrecer

GymProgress es una aplicación móvil nativa para Android respaldada por una API REST en Spring Boot y una base de datos MySQL. SE ofrece como un servicio digital (Software as Service – SaaS) bajo un modelo **Freemium**.

- ✓ **Servicio base (Gratuito):** Permite al usuario registrar sus entrenamientos diarios, crear rutinas personalizadas y hacer un seguimiento básico de variables como series, repeticiones (RPE) y peso (KG).
- ✓ **Servicio Premium:** Funcionalidades avanzadas como el análisis gráfico del progreso largo plazo, métricas detalladas de volumen de entrenamiento, y acceso al “Catálogo de Rutinas” curado por entrenadores.
- ✓

Clientes o consumidores principales

El público objetivo se divide en dos segmentos:

- ✓ **Cliente Final:** Usuario de gimnasio, mayoritariamente entre 16 y 35 años, nativos digitales, que entrenan de forma recurrente y necesitan llevar un control riguroso de sus levantamientos. Son usuarios que valoran la rapidez y una interfaz que no moleste durante el entrenamiento.

- ✓ **Cliente Profesional:** Entrenadores personales que podrían utilizar el “Panel de Administrador” de la plataforma para volcar rutinas predefinidas y monitorizar a distancia a sus clientes.

Ventajas y desventajas competitivas

- ✓ **Ventajas competitivas**
 - Usabilidad específica: Interfaz “Fatigue-Proof” (botones grandes, tema oscuro, flujos rápidos) diseñada específicamente para usarse mientras se descansa entre series, superando a las apps genéricas que requieren demasiados clics.
 - Rendimiento: Al estar desarrollada de forma nativa en Kotlin, la fluidez y el consumo de batería están mucho más optimizadas que en alternativas híbridas.
- ✓ **Desventajas competitivas**
 - Mercado saturado: Existe competidores gigantes y muy asentados con grandes comunidades detrás.
 - Limitación de plataforma: En su lanzamiento inicial, la aplicación solo estará disponible para Android, perdiendo el nicho de mercado de usuarios de iOS.
 - Falta de capital inicial: Al ser un proyecto independiente, no hay un gran presupuesto para campañas de marketing agresivas.

Costes implicados, ingresos esperados, margen, beneficio y el tiempo.

- ✓ **Tiempo de ejecución:** Fase de desarrollo 3-5 meses
 - Fase de pruebas (Beta) y subida a tiendas: 1 mes.
 - Fase de monetización: A partir del mes 6 de vida de la aplicación.
- ✓ **Costes implicados (Estimación primer año):**
 - Licencias: Cuenta de desarrollador de Google Play (pago único de 25€).
 - Infraestructura: Servidor VPS para alojar Spring Boot + MySQL y dominio (Aprox. 20/30€/mes)
 - Coste de desarrollo: Como emprendedor, el coste principal es el tiempo invertido (aprox. 300 horas). Aun precio de desarrollador junior (15€/h), supondría un coste de oportunidad de 4.500€, aunque inicialmente no se desembolsa.
- ✓ **Ingresos esperados, margen y beneficio:**
 - El modelo de ingresos se basa en una suscripción Pro de 2,99€/mes o 24,99€/año.

- Con un objetivo realista de conseguir 100 usuarios Premium en el primer año, los ingresos brutos rondarían los 2500€.
- Al ser un producto de software, el margen bruto es muy alto (cercando al 85%, descontando comisiones de las pasarelas de pago y de Google Play).
- Beneficio: El primer año el beneficio económico directo sería modesto (Ingresos 2.500€ - Costes fijos 385€ = 2.115€), pero el retorno real se encuentra en la validación del mercado y el valor del producto creado.